

Framework Selection Report

Pangasinan Heritage Digital Showcase — Deliverable 1.1

Prepared by: Lastname, Firstname

Instructor: Rey Molano

1. Selected Framework

This project selects Option A — React with Next.js 14 (App Router) — as the framework for the Pangasinan Heritage Digital Showcase, over Option B (Vue with Nuxt.js 3). The sections below justify this decision using quantitative and qualitative comparison across the criteria required by the activity brief.

2. Comparison Criteria

Criterion	Next.js 14 (App Router)	Nuxt.js 3
Bundle size / performance	Ships a small React runtime baseline; App Router adds React Server Components, which send zero client JS for server-only pieces (e.g. static heritage descriptions). Automatic image optimization and route-level code-splitting keep first-load JS low on 3G/4G.	Vue's runtime is already lighter than React's on paper, and Nuxt 3 (Vite-based) produces small bundles too. Nitro's hybrid rendering helps, but Nuxt does not yet ship a server-components model as integrated as RSC, so more interactive-page JS reaches the client by default.
Developer velocity	File-based routing, built-in layouts, and a single mental model (React) speed up building the required Header, Grid, and Card organisms. Hot reload and TypeScript support are mature.	Also file-based routing with auto-imported components, arguably even less boilerplate for small teams. Vite's dev server is very fast. Slightly faster to scaffold, but the gap is small for a project this size.
Ecosystem maturity	Largest component/library ecosystem of any frontend framework (headless UI kits, form libraries, testing tools). Vercel backs Next.js directly, and most static-site-generation tutorials/examples target it.	Growing steadily and well maintained, but the pool of ready-made Vue 3 / Nuxt 3 component libraries and Stack Overflow answers is noticeably smaller than React's.
Learning curve	The team already has JavaScript/JSX exposure from other coursework, so React's component model is familiar. App Router's server/client component split is a new concept to learn.	Vue's template syntax (v-if, v-for, single-file components) is arguably gentler for beginners, but switching mental models mid-course has its own cost.
Component architecture fit	React's composition model (props + children) maps directly onto Atomic Design: atoms and molecules are plain function components, organisms compose them with no extra abstraction layer needed.	Vue's Single File Components (template/script/style in one file) also map cleanly onto Atomic Design, and scoped styles are built in without needing a CSS-in-JS or utility-class layer.
Documentation & community support	Extensive official docs, a large Discord/GitHub community, and frequent long-term-support releases.	Solid official docs and an active but smaller community than React's.
Suitability for this project	Static/JAMstack export, mobile-first responsive design, and WCAG-friendly semantic HTML are all first-class, well-documented Next.js patterns.	Equally capable of static/JAMstack output via Nuxt's generate command; no functional gap for this project's requirements.

3. Quantitative Benchmark Snapshot

Public benchmark data (published framework comparison suites and Next.js/Nuxt release notes, accessed for this report) points to the following approximate figures for a comparable marketing/showcase site with 3 routes and image-heavy cards:

- Baseline client JS on first load: Next.js 14 App Router $\approx$ 80–100 KB gzipped when server components are used for static content; a fully client-rendered Nuxt 3 page in the same range ($\approx$ 90–110 KB) unless Nitro island/partial hydration is deliberately configured.
- Lighthouse mobile performance score (throttled 4G, similar hosting): both frameworks reach 90+ when images are optimized and fonts are system/local — the frameworks are not the bottleneck; asset discipline is.
- Time-to-first-contentful-paint on simulated 3G: both frameworks perform comparably once static export/ISR is used; the deciding factor for this report is ecosystem and team familiarity rather than a clear performance gap.

In short: performance is roughly a tie when both frameworks are configured correctly for static/JAMstack deployment. The deciding factors are ecosystem maturity, documentation depth, and fit with the team's existing React knowledge — all of which favor Next.js for this specific team and timeline.

4. Why Next.js 14 Fits the Pangasinan Heritage Digital Showcase

- Lightning-Fast: App Router's React Server Components let static heritage-site content (descriptions, images) ship with zero extra client JavaScript, which matters directly for the brief's 3G/4G requirement.
- Mobile-First: Next.js's built-in Image component (used by the Image atom in this project) automatically serves responsive, lazy-loaded images sized to the viewport.
- Maintainable: The file-based App Router plus the atoms/molecules/organisms folder structure keeps components decoupled and independently updatable, matching the brief's modular-architecture requirement.
- Accessible: Next.js does not fight semantic HTML or ARIA attributes, so the WCAG 2.1 AA patterns used in this project's components (focus rings, skip links, aria-expanded menus) work exactly as written.
- Deployable: `next build` & `next export`-style static output (or Vercel's static/ISR hosting) satisfies the JAMstack deployment requirement without a custom Node server.

5. Conclusion

Both frameworks could satisfy the Pangasinan Heritage Digital Showcase requirements competently. Next.js 14 is selected because it gives the largest ecosystem and documentation base to draw on while building the Atomic Design System (Deliverable 1.2), aligns with the team's existing React knowledge, and its App Router/Server Components model directly targets the brief's mobile-data-performance requirement without extra configuration.